

Textbook: Pathfinding

Introduction to Pathfinding

Pathfinding is the process of finding the shortest or most efficient route between two points. In computer science, it's commonly used in game development, robotics, and mapping applications. We'll explore pathfinding in a grid-based environment, which is a common scenario in games and simulations.

Primitives of Pathfinding

- Grid-Based Environment:** Your environment is divided into a grid, where each cell in the grid is a potential step in a path.
- Nodes and Neighbors:** Each cell in the grid is a node. Pathfinding involves moving from one node to its neighboring nodes until the target is reached.
- Algorithm Choice:** Many algorithms exist for pathfinding, like A*, Dijkstra's, and Breadth-First Search (BFS). We'll start with a simple approach, often referred to as Flood Fill, which is similar to BFS.

PATHFINDING

Pathfinding is a computational problem solving algorithm used to find the shortest and most efficient path between two points in a space. When implemented on a grid, it's a crucial element of many applications, including robotics, computer games, and navigation systems. When applied to the two-dimensional navigation through environments in an optimal manner.

Common pathfinding algorithms include A*, Dijkstra's Algorithm, and Breadth-First Search. Each of these algorithms has its own strengths and weaknesses, making them suitable for recovering the most efficient path.

In our examples, we will use an **Environment class** and a **Tile class**. The environment is grid will be responsible for the pathfinding calculation, and the tile will offer methods to help find neighbors and store temporary data that will be needed in the calculation.

Accessing Neighbors in a Grid

- Neighbor Definition:** In a 2D grid, a cell typically has up to 8 neighbors (up, down, left, right, and diagonal). For simplicity, we'll start with 4 neighbors (up, down, left, right).
- Getting Neighbors:** For a node at (x, y), its neighbors are (x-1, y), (x+1, y), (x, y-1), and (x, y+1).

ACCESSING NEIGHBORS

```
def get_neighbors(self):
    neighbors = []
    for dx, dy in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        nx, ny = self.x + dx, self.y + dy
        if 0 <= nx < self.grid.width and 0 <= ny < self.grid.height:
            neighbors.append((nx, ny))
    return neighbors
```

Let's look at the neighbor access functions inside the Tile class:

```
1 def get_neighbors(self):
2     # Initialize an empty list to store neighboring cells
3     neighbors = []
4
5     # Check if the left neighbor exists (making sure the cell is not on the left edge)
6     if self.x > 0:
7         # Append the left neighbor to the neighbors list
8         neighbors.append((self.x-1, self.y))
9
10    # Check if the right neighbor exists (making sure the cell is not on the right edge)
11    if self.x < self.grid.width - 1:
12        # Append the right neighbor to the neighbors list
13        neighbors.append((self.x+1, self.y))
14
15    # Check if the upper neighbor exists (making sure the cell is not on the top edge)
16    if self.y > 0:
17        # Append the upper neighbor to the neighbors list
18        neighbors.append((self.x, self.y-1))
19
20    # Check if the lower neighbor exists (making sure the cell is not on the bottom edge)
21    if self.y < self.grid.height - 1:
22        # Append the lower neighbor to the neighbors list
23        neighbors.append((self.x, self.y+1))
24
25    # Return the list of neighboring cells
26    return neighbors
27
```

The function is part of a class that represents a cell in a grid. It finds and returns the adjacent cells of the current cell.

- Neighbor Checking:** The method uses checks to ensure it doesn't try to access neighbors outside the grid boundaries. For example, it checks if `self.x > 0` before trying to access the left neighbor.
- 4-Way Connectivity:** The function considers four-way connectivity (up, down, left, right). It doesn't include diagonal neighbors, but they could be added with additional checks.
- Returning Neighbors:** Neighbors are appended to `neighbors`, which is a list (for array) representing all cells in the grid. The return `neighbors` is used to access the correct neighboring cells.

Flood Fill for Pathfinding

- Basic Concept:** Start from the initial node and "flood" out to neighboring nodes, then to their neighbors, and so on, until the target is reached.
- Implementation:** Use a queue to manage nodes to visit. Initially, enqueue the starting node. Then, repeatedly dequeue a node, enqueue its unvisited neighbors, and mark them as visited.

FLOOD FILL CALCULATION

```
def flood_fill(self, start_x, start_y):
    queue = deque([(start_x, start_y)])
    visited = set()
    path = []

    while queue:
        x, y = queue.popleft()
        if (x, y) in visited:
            continue
        visited.add((x, y))
        path.append((x, y))

        for nx, ny in self.get_neighbors():
            if not visited.add((nx, ny)):
                queue.append((nx, ny))
```

Let's look at the Flood Fill calculation within the Environment class:

```
1 def flood_fill(self):
2     # Check if the flood process is still running
3     if self.running:
4         # Check if there are cells left to be processed
5         if len(self.queue) > 0:
6             # Dequeue the first cell from the queue and work on it
7             x, y = self.queue.popleft()
8             # Mark the current cell as visited
9             self.visited.add((x, y))
10
11            # Check if the current cell is the goal or end node
12            if (x, y) == self.target:
13                print("Goal reached!")
14                # Stop the flood fill process
15                self.running = False
16                # Return the path found
17                return self.path
18
19            # Enqueue all unvisited neighbors of the current cell
20            for nx, ny in self.get_neighbors():
21                if not visited.add((nx, ny)):
22                    self.queue.append((nx, ny))
23
24            # Mark the current cell as the parent of the next cell
25            self.parent[(x, y)] = (x, y)
26
27            # Add the neighbor to the queue for further processing
28            self.queue.append((nx, ny))
29
30            # Mark the current cell as the parent of the neighbor for path reconstruction
31            self.parent[(nx, ny)] = (x, y)
```

- Running Check:** The algorithm runs as long as self.running is True.
- Queue:** A queue which acts as a stack in this context holds cells to be visited. The algorithm continuously removes and processes cells from the stack.
- Visited Cells:** Each cell visited is marked as such to avoid reprocessing the same cell.
- Goal Check:** If the current cell is the goal or end node, the algorithm prints a message, stops running, marks the path as found, and reconstructs the path.
- Processing Neighbors:** The algorithm then checks each neighbor of the current cell. If a neighbor hasn't been visited and isn't already in the queue, it's added to the queue for future processing, and its parent is set to the current cell. This parent link is essential for path reconstruction.
- Path Reconstruction:** The reconstruct_path method not shown here (it's in the next cell) traces back from the end node to the start node using the parent links, creating the path the algorithm found.

Reconstructing the Optimal Path

- Tracking Path:** As you visit each node, store where you came from. This way, you can trace back from the target to the start.
- Reconstruction:** Once the target is found, follow the stored paths backward to the start, which gives you the optimal path.

OPTIMAL PATH

```
def reconstruct_path(self, current_cell):
    path = []
    while current_cell.parent is not None:
        path.append(current_cell)
        current_cell = current_cell.parent
    path.append(start_node)
    path.reverse()
    return path
```

Let's look at the reconstruct_path method to understand how it reconstructs the path:

```
1 def reconstruct_path(self, current_cell):
2     # Verify the start of path reconstruction
3     path = []
4
5     # Loop back from the current cell (typically the end node) to the start node
6     while current_cell.parent is not None:
7         # Add the current cell to the path list
8         path.append(current_cell)
9         # Move to the parent of the current cell
10        current_cell = current_cell.parent
11
12    # Mark the start of path reconstruction is completed
13    print("Path")
14
15    # Add the start node to the path list
16    path.append(start_node)
17
18    # Sort the path by index
19    path.reverse()
20
21    # Verify the end of path reconstruction
22    print("Reconstruction done!")
23
```

The function is used to trace and construct the path found by the pathfinding algorithm. It works backward from the end node to the start node.

- Path Tracing:** Starting from the end node (`current_cell`), it iteratively traces back to the parent of each cell. This parent-child relationship is typically established during the pathfinding process.
- Appending to Path:** Each cell visited during this backward traversal is added to `path`, which is a list that will store the final path.
- Start Node Inclusion:** Finally, the start node is added to the path list. This is important as the loop terminates once the parent of the current cell is None, which would be the case for the start node.
- Reversing:** The path is reversed because the path was traced backward from the end node to the start node.

Go to next slide

Completed

Like Dislike Report an issue